

TELE-SEVERINO PROJETO INTEGRADOR – BLOCO 4: DESENVOLVIMENTO

O Tele Severino é uma plataforma digital criada para conectar, de forma rápida e acessível, pessoas que possuem dúvidas técnicas ou práticas a profissionais experientes de diversas áreas, por meio de atendimento remoto via vídeo ou áudio, com cobrança por minuto de uso.

INTEGRANTES DO GRUPO 13

Data de realização: 25/03/2026

Luciano S. da Silva Junior—RA: 926114218—lucianososiljunior@uni9.edu.br

Israel de Albuquerque — RA: 926116180 — israelalbuquerque@uni9.edu.br

Edvanio Maximo da Silva — RA: 926114228 — silva.edvanio@uni9.edu.br

Roger Sousa Funaki — RA: 926115899 — roger.funaki@uni9.edu.br

Cicero Silva de Almeida Junior — RA: 1726104302 — cj.almeida@uni9.edu.br

PADRÕES DE PROJETO (GoF)

Durante o desenvolvimento do sistema Tele Severino, foram aplicados padrões de projeto (Design Patterns) descritos por Gamma, Helm, Johnson e Vlissides (GoF – Gang of Four). A utilização desses padrões teve como objetivo aumentar a reutilização de código, facilitar a manutenção do sistema e promover uma arquitetura mais organizada e escalável. Os padrões selecionados foram:

- Singleton (Criacional);
- Factory Method (Criacional);
- Strategy (Comportamental).

PADRÃO SINGLETON

Categoria: Criacional.

Problema Resolvido: O sistema necessita realizar diversas operações de acesso ao banco de dados, como cadastro de usuários, autenticação, cadastro de especialistas e consultas. A criação indiscriminada de conexões pode gerar desperdício de recursos e dificultar o gerenciamento da aplicação.

Para solucionar esse problema foi utilizado o padrão Singleton, garantindo a existência de uma única instância responsável pelo gerenciamento da conexão com o banco de dados.

Aplicação no Projeto: O padrão foi aplicado na camada de acesso a dados, centralizando a criação da conexão utilizada pelas rotas da API.

Diagrama UML:

```
+-----+
| DatabaseSingleton |
+-----+
| - instancia |
+-----+
| + getInstance() |
| + conectar() |
+-----+
```

Exemplo de Código:

```
class DatabaseSingleton:
    _instancia = None
    @staticmethod
    def get_instancia():
        if DatabaseSingleton._instancia is None:
            DatabaseSingleton._instancia = DatabaseSingleton()
        return DatabaseSingleton._instancia
```

BENEFÍCIOS OBTIDOS:

- Controle centralizado da conexão;
- Redução do consumo de recursos;
- Facilidade de manutenção.

PADRÃO FACTORY METHOD

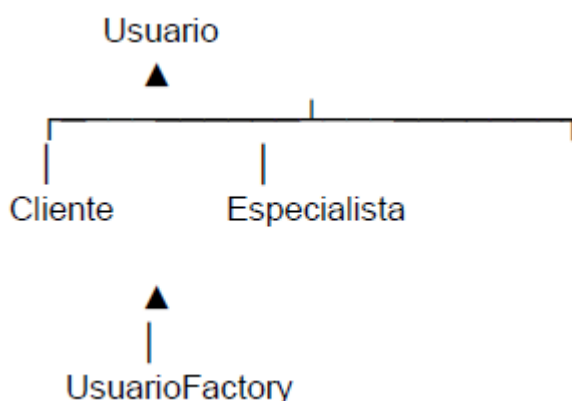
Categoria: Criacional.

Problema Resolvido: O sistema trabalha com diferentes tipos de usuários, como clientes e especialistas. A criação desses objetos diretamente na aplicação poderia gerar alto acoplamento e dificultar futuras expansões.

O padrão Factory Method foi utilizado para centralizar a criação dos diferentes tipos de usuários.

Aplicação no Projeto: Durante o cadastro, a fábrica é responsável por identificar o tipo de usuário e instanciar a classe correspondente.

Diagrama UML:



Exemplo de Código

```
class UsuarioFactory:
    @staticmethod
    def criar_usuario(tipo):
        if tipo == "CLIENTE":
            return Cliente()
        elif tipo == "ESPECIALISTA":
            return Especialista()
```

BENEFÍCIOS OBTIDOS:

- Baixo acoplamento;
- Facilidade para adicionar novos tipos de usuários;
- Centralização da lógica de criação.

PADRÃO STRATEGY

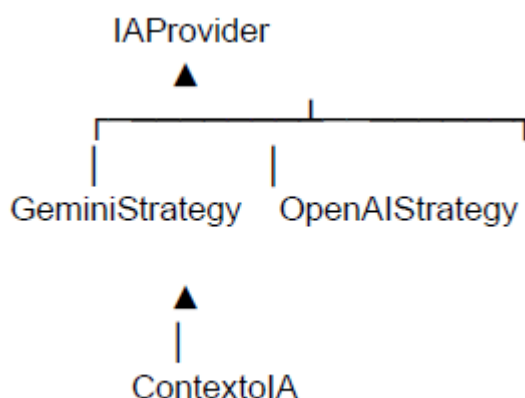
Categoria: Comportamental.

Problema Resolvido: O módulo de Inteligência Artificial do Tele Severino pode futuramente utilizar diferentes provedores de IA, como Gemini, OpenAI ou Claude. Sem um padrão adequado, seria necessário alterar diversas partes do sistema para cada nova integração.

Para evitar esse problema foi utilizado o padrão Strategy.

Aplicação no Projeto: O padrão permite encapsular diferentes algoritmos de geração de resposta em estratégias independentes, possibilitando a troca do provedor sem modificar a lógica principal da aplicação.

Diagrama UML



Exemplo de Código

```
class IAProvider:
def responder(self, pergunta):
pass
class GeminiStrategy(IAProvider):
def responder(self, pergunta):
return "Resposta gerada pelo Gemini"
class ContextoIA:
def __init__(self, estrategia):
self.estrategia = estrategia
def responder(self, pergunta):
return self.estrategia.responder(pergunta)
```

BENEFÍCIOS OBTIDOS:

- Flexibilidade para trocar provedores de IA;
- Maior organização do código;
- Facilidade para manutenção e expansão futura.

A utilização dos padrões Singleton, Factory Method e Strategy contribuiu para uma arquitetura mais organizada e preparada para futuras evoluções do sistema Tele Severino.

Os padrões escolhidos atenderam às necessidades específicas do projeto, promovendo reutilização de código, redução de acoplamento e maior flexibilidade para futuras implementações.

A adoção dessas práticas demonstra a aplicação de conceitos de Engenharia de Software



voltados à construção de sistemas escaláveis e de fácil manutenção.

DOCUMENTAÇÃO DE APIs

A API do projeto Tele Severino foi desenvolvida utilizando FastAPI. Essa tecnologia gera automaticamente uma documentação interativa no formato Swagger/OpenAPI, permitindo visualizar, testar e validar os endpoints diretamente pelo navegador.

LINKS DA DOCUMENTAÇÃO DA API

Swagger UI:

<http://127.0.0.1:8000/docs>

OpenAPI JSON:

<http://127.0.0.1:8000/openapi.json>

ReDoc:

<http://127.0.0.1:8000/redoc>

Para utilizar esses links, o backend precisa estar rodando localmente:

```
cd backend
uvicorn main:app --reload
```

POSTMAN COLLECTION

Também foi criada uma collection para execução dos testes de API no Postman, junto com um environment local.

Esses arquivos podem ser importados no Postman para testar os endpoints com a base URL local do projeto.

Collection: `Tele_Severino_API_FastAPI.postman_collection.json`

Environment: `Tele_Severino_Local.postman_environment.json`

DESCRIÇÃO GERAL DA API

A API permite o cadastro e autenticação de usuários, gerenciamento de especialistas e utilização de um recurso de Inteligência Artificial como prova de conceito. As rotas estão organizadas em três grupos principais:

- Usuários;
- Especialistas;
- Inteligência Artificial.



RESUMO DOS ENDPOINTS

MÉTODO	ENDPOINT	FINALIDADE
POST	/usuarios	Criar/cadastrar usuario
POST	/login	Realizar login
PUT	/usuario/{id}	Atualizar usuario
DELETE	/delete_usuario/{id}	Excluir usuario
POST	/especialistas/	Cadastrar especialista
GET	/especialistas/	Listar especialistas
GET	/especialistas/{id_especialista}	Buscar especialista por ID
PUT	/especialistas/{id_especialista}	Atualizar especialista
DELETE	/especialistas/{id_especialista}	Excluir especialista
POST	/ia	Enviar pergunta para a IA

CRIAR USUÁRIO

Método / POST / Endpoint / usuarios

Endpoint responsável por cadastrar um novo usuario no sistema

Exemplo de request:

```
{
  "nome": "Roger",
  "email": "roger@teste.com",
  "senha": "123456",
  "tipo": "CLIENTE"
}
```

Exemplo de response esperado:

```
{
  "mensagem": "Usuário cadastrado com sucesso"
}
```

Códigos de status:

- 200 - Usuário cadastrado com sucesso;
- 400 - Dados inválidos ou e-mail já cadastrado;
- 500 - Erro interno do servidor.

LOGIN DE USUÁRIO

Método / POST / Endpoint / login



Endpoint responsável por autenticar um usuário cadastrado no sistema.

Exemplo de request:

```
{
  "email": "roger@teste.com",
  "senha": "123456"
}
```

Exemplo de response esperado:

```
{
  "mensagem": "Login realizado com sucesso"
}
```

Códigos de status

- 200 - Login realizado com sucesso;
- 401 - E-mail ou senha inválidos;
- 500 - Erro interno do servidor.

ATUALIZAR USUÁRIO

Método / PUT / **Endpoint** / usuario/{id}

Endpoint responsável por atualizar os dados de um usuário existente.

Parâmetros:

id - Identificador do usuário

Exemplo de request:

```
{
  "nome": "Roger Atualizado",
  "email": "roger.atualizado@teste.com",
  "senha": "123456",
  "tipo": "CLIENTE"
}
```

Exemplo de response esperado:

```
{
  "mensagem": "Usuário atualizado com sucesso"
}
```

Códigos de status:

- 200 - Usuário atualizado com sucesso;
- 404 - Usuário não encontrado;
- 500 - Erro interno do servidor.

EXCLUIR USUÁRIO

Método / DELETE / **Endpoint** / delete_usuario/{id}





Endpoint responsável por excluir um usuário cadastrado no sistema.

Parâmetros

id - Identificador do usuário.

Request:

Não possui body.

Exemplo de response esperado:

```
{
  "mensagem": "Usuário excluído com sucesso"
}
```

Códigos de status:

- 200 - Usuário excluído com sucesso;
- 404 - Usuário não encontrado;
- 500 - Erro interno do servidor.

CADASTRAR ESPECIALISTA

Método / POST / **Endpoint** / especialistas/

Endpoint responsável por cadastrar um especialista no sistema. O especialista representa o profissional que poderá prestar atendimento dentro da plataforma Tele Severino.

Exemplo de request:

```
{
  "id_especialista": 1,
  "especialidade": "Consultoria Doméstica",
  "valor minuto": 2.5
}
```

Exemplo de response esperado:

```
{
  "mensagem": "Especialista cadastrado com sucesso"
}
```

Códigos de status:

- 200 - Especialista cadastrado com sucesso;
- 400 - Dados inválidos;
- 500 - Erro interno do servidor.

LISTAR ESPECIALISTAS

Método / GET / **Endpoint** /especialistas/

Endpoint responsável por retornar todos os especialistas cadastrados no sistema.





Request:

Não possui body.

Exemplo de response esperado:

```
[
{
  "id_especialista": 1,
  "especialidade": "Consultoria Doméstica",
  "valor minuto": 2.5
}
]
```

Códigos de status:

- 200 - Lista de especialistas retornada com sucesso;
- 500 - Erro interno do servidor.

BUSCAR ESPECIALISTA POR ID

Método / GET / **Endpoint** / especialistas / {id_especialista}

Endpoint responsável por buscar os dados de um especialista específico.

Parâmetros:

id_especialista - Identificador do especialista.

Request:

Não possui body.

Exemplo de response esperado:

```
{
  "id_especialista": 1,
  "especialidade": "Consultoria Doméstica",
  "valor minuto": 2.5
}
```

Códigos de status:

- 200 - Especialista encontrado com sucesso;
- 404 - Especialista não encontrado;
- 500 - Erro interno do servidor.

ATUALIZAR ESPECIALISTA

Método / PUT / **Endpoint** / especialistas / {id_especialista}

Endpoint responsável por atualizar os dados de um especialista existente.

Parâmetros:





id_especialista - Identificador do especialista.

Exemplo de request:

```
{
"especialidade": "Consultoria Residencial",
"valor_minuto": 3.0
}
```

Exemplo de response esperado:

```
{
"mensagem": "Especialista atualizado com sucesso"
}
```

Códigos de status:

200 - Especialista atualizado com sucesso;
404 - Especialista não encontrado;
500 - Erro interno do servidor.

EXCLUIR ESPECIALISTA

Método / DELETE / Endpoint / especialistas /{id_especialista}

Endpoint responsável por excluir um especialista cadastrado.

Parâmetros

id_especialista - Identificador do especialista

Request

Não possui body.

Exemplo de response esperado:

```
{
"mensagem": "Especialista excluído com sucesso"
}
```

Códigos de status:

- 200 - Especialista excluído com sucesso;
- 404 - Especialista não encontrado;
- 500 - Erro interno do servidor.

ENVIAR PERGUNTA PARA INTELIGÊNCIA ARTIFICIAL

Método / POST / Endpoint / ia

Endpoint responsável por enviar uma pergunta para o recurso de Inteligência Artificial da aplicação.





Exemplo de request:

```
{  
  "pergunta": "Como posso organizar melhor minha casa?"  
}
```

Exemplo de response esperado:

```
{  
  "resposta": "Sugestão gerada pela inteligência artificial."  
}
```

Códigos de status:

- 200 - Resposta gerada com sucesso;
- 400 - Pergunta invalida;
- 500 - Erro interno do servidor.

INTELIGENCIA ARTIFICIAL NA APLICAÇÃO

No projeto Tele Severino, a Inteligência Artificial será utilizada como um recurso de apoio ao usuário, funcionando como uma ferramenta de orientação rápida dentro da plataforma. A proposta é permitir que o usuário envie uma dúvida ou solicitação e receba uma resposta automatizada relacionada ao contexto do serviço desejado. Essa resposta pode ajudar o usuário antes de escolher um especialista ou iniciar uma chamada.

Exemplos de uso da IA no sistema:

- Recomendação de serviços;
- Sugestão de especialistas;
- Apoio em dúvidas simples;
- Chatbot de atendimento inicial;
- Orientações antes de iniciar uma chamada;
- Automação de respostas para perguntas frequentes.

Exemplo de uso prático:

Um usuário pode perguntar:

"Como posso organizar melhor minha casa?"

A IA pode retornar uma orientação inicial, como:

"Você pode começar separando os ambientes por prioridade, removendo objetos sem uso e organizando os itens por categoria."

Depois disso, o usuário pode escolher um especialista da plataforma para receber uma consultoria mais detalhada por chamada.

Ferramenta/API de IA escolhida





Para a prova de conceito do projeto Tele Severino, foi planejada a utilização de uma API de Inteligência Artificial baseada em modelo de linguagem, como OpenAI API ou Gemini API.

A escolha desse tipo de ferramenta se justifica pela capacidade de interpretar perguntas em linguagem natural e gerar respostas automáticas, facilitando o atendimento inicial do usuário.

Justificativa da IA

- Atendimento inicial automatizado;
- Redução de dúvidas simples antes da chamada;
- Apoio na recomendação de especialistas;
- Melhoria na experiência do usuário;
- Maior agilidade no processo de escolha do serviço.

Prova de conceito da IA

A prova de conceito foi implementada através do endpoint POST /ia. Esse endpoint recebe uma pergunta enviada pelo usuário e retorna uma resposta simulada ou gerada pela IA.

POST /ia

Request:

```
{
  "pergunta": "Qual especialista devo procurar para organizar minha casa?"
}
```

Response:

```
{
  "resposta": "Você pode procurar um especialista em organização residencial ou consultoria doméstica."
}
```

Essa funcionalidade demonstra como a IA pode ser integrada ao sistema para apoiar o usuário antes do atendimento com um especialista.

CHECKPOINT 2 - ESTADO ATUAL DO PROJETO

O projeto Tele Severino encontra-se em fase de desenvolvimento e integração das principais funcionalidades previstas.

Progresso geral do projeto:

- Definição da ideia principal do sistema;
- Criação do fluxo de navegação da aplicação;
- Desenvolvimento das telas principais;
- Criação do backend com FastAPI;
- Criação das rotas de usuários;
- Criação das rotas de especialistas;
- Criação do endpoint de Inteligência Artificial;





- Documentação da API com Swagger/OpenAPI;
- Criação da collection do Postman;
- Execução local do backend;
- Integração inicial entre frontend e backend.

Funcionalidades concluídas:

- Cadastro de usuário;
- Login de usuário;
- Atualização de usuário;
- Exclusão de usuário;
- Cadastro de especialista;
- Listagem de especialistas;
- Busca de especialista por ID;
- Atualização de especialista;
- Exclusão de especialista;
- Endpoint de Inteligência Artificial;
- Documentação Swagger automática;
- Collection Postman para testes.

Funcionalidades em andamento:

- Melhoria da interface visual;
- Ajustes de integração com o frontend;
- Melhoria no fluxo de chamada entre usuário e especialista;
- Ajustes no fluxo de pagamento;
- Melhoria na tela de avaliação;
- Organização final da documentação;
- Atualização do repositório GitHub;
- Atualização do quadro Kanban.

KANBAN DO PROJETO

Link do *GitHub Projects*

<https://github.com/users/CiriloDevelope/projects/1> Colunas utilizadas: A Fazer, Em Progresso e Concluído.

Repositório *GitHub*

Link do repositório: <https://github.com/CiriloDevelope/Tele-Severino>

URL SSH: <git@github.com:CiriloDevelope/Tele-Severino.git>

INSTRUÇÕES DE USO

Clonar o repositório

- `git clone git@github.com:CiriloDevelope/Tele-Severino.git`
- Entrar na pasta do projeto: `cd Tele-Severino`
- Abrir o projeto em editores como Visual Studio Code, IntelliJ IDEA ou Android Studio.

